

实验报告

课程: 机器学习	实验名称: 多项式回归	日期: 3.23
姓名: 雷文豪	学号: 138022024031	专业班级: 人工智能

1 实验目的

1. 掌握非线性回归算法的实现，学会模拟数据生成、导入数据集和结果可视化的方法。
2. 学习解题设计、通过代码编写与运行实施解题、完成程序正确性的验证。

2 实验内容提要 with 实验设备

1. 给待解问题设计求解方法和回归模型。
2. 对给定数据集，设计回归公式的求解方法、编写程序实施求解。
3. 实验结果的分析与确认。
4. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 实验内容与步骤

3.1 变量间非线性关系问题（掌握二次多项式回归分析原理）

假设某数据集中的因变量 Y 与自变量 X 之间存在近似平方的关系，请设计合适的回归模型表达该数据集所隐含的这种关系，并用模拟数据验证（即，使用模拟数据求解出你设计的回归公式，查看公式中的因变量与自变量之间是否为平方关系）。

3.1.1 工作任务一：制造模拟数据

提示：制造符合题目的模拟数据集 D 之后，**忘掉制作方法**，假装模拟数据集 D 是老师给的，你对数据集 D 一无所知。

制造方法： 用如下二次多项式回归模型制造模拟数据：

$$y = ax^2 + bx + c + \epsilon,$$

其中 $a = 0.9, b = 0.1, c = 2$, ϵ 属于噪声。

Python 代码： 抄写如下代码，制造出模拟数据集 $D = \{x_i, y_i\}$ ，其中 x_i 对应 dataX 数据， y_i 对应 dataY 数据；公式中的噪声 ϵ 用 `random.rand()` 实现。

```
1 import numpy as np
2 # 数据点的数量
3 n = 100
4 # 产生自变量 x 的数据，有波动的 [-1,9] 范围值
5 dataX = 10 * np.random.rand(n, 1) - 1
6 # 产生因变量 y 的数据
7 dataY = 0.9 * dataX ** 2 + 0.1 * dataX + 2 + 5 * np.random.rand(n, 1)
```

3.1.2 工作任务二：给数据集 D 挑选一个回归公式，建模描述数据中隐含关系

3.1.3 工作任务三：使用数据集 D 求解回归公式的回归系数

- 简单写出多项式回归的求解步骤（例如 $y = 9.9x^2 + 8.8x + 7.7$ ）。禁止复制粘贴、禁止使用截图，可以不用公式编辑器，用字母和上下标符号。可参考课本或资料。
- 获得的结果，即写出有回归系数值的回归公式（例如 $y = 9.9x^2 + 8.8x + 7.7$ ）。
- 程序运行成功的截图（包含源文件名 <字母 + 学号后 3 位>、运行光标在程序尾行）。

（1）求解步骤

1. 按照要求生成模拟数据，按照公式 $y = 0.9x^2 + 0.1x + 2 + \text{噪声}$ ，生成数据集
2. 把 x^2 、 x 、常数 1 拼成特征矩阵，把非线性问题转为线性问题，完成特征构造
3. 使用最小二乘法求解出系数 w_2 、 w_1 、 w_0
4. 得到回归模型 $y = w_2x^2 + w_1x + w_0$
5. 将结果可视化，看拟合效果

（2）源代码

3.1 二次多项式回归分析

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# 任务一：制造模拟数据集D
```

```
# 固定随机种子，保证结果可复现
```

```
np.random.seed(42)
```

```

# 样本数量
n = 100

# 生成自变量x
dataX = 10 * np.random.rand(n, 1) - 1

# 生成因变量y
dataY = 0.9 * dataX ** 2 + 0.1 * dataX + 2 + 5 * np.random.rand(n, 1)

# 任务二：设计二次多项式回归模型
# 模型形式：  $y = w_2 * x^2 + w_1 * x + w_0$ 
# 构造增广特征矩阵X，每一行是  $[x^2, x, 1]$ 
X = np.hstack([dataX ** 2, dataX, np.ones((n, 1))])
Y = dataY

# 任务三：求解回归系数
XTX = X.T @ X

# 矩阵求逆
XTX_inv = np.linalg.inv(XTX)

# 计算最终系数
W = XTX_inv @ X.T @ Y

# 提取拟合得到的系数
w2, w1, w0 = W.flatten()

# 结果输出与验证
print("3.1 二次多项式回归分析")
print(f"拟合得到的回归公式：  $y = \{w2:.2f\} * x^2 + \{w1:.2f\} * x + \{w0:.2f\}$ ")
print(f"原始真实公式：  $y = 0.90 * x^2 + 0.10 * x + 2.00$ ")

# 结果可视化
# 生成用于绘制拟合曲线的连续x值
x_plot = np.linspace(-1, 9, 100).reshape(-1, 1)

# 计算拟合曲线的y值
y_plot = w2 * x_plot ** 2 + w1 * x_plot + w0

# 绘制图像
plt.rcParams['font.sans-serif'] = ['SimHei'] # 解决中文显示

```

```
plt.rcParams['axes.unicode_minus'] = False
plt.figure(figsize=(10, 6))
# 原始数据散点图
plt.scatter(dataX, dataY, color='steelblue', label='原始带噪声数据', alpha=0.7)
# 拟合曲线
plt.plot(x_plot, y_plot, color='crimson', linewidth=2, label='二次多项式拟合曲线')
# 图像标注
plt.xlabel('自变量 X', fontsize=12)
plt.ylabel('因变量 Y', fontsize=12)
plt.title('3.1 二次多项式回归拟合结果', fontsize=14)
plt.legend(fontsize=12)
plt.grid(alpha=0.3)
plt.show()
```

(3) 运行结果与分析

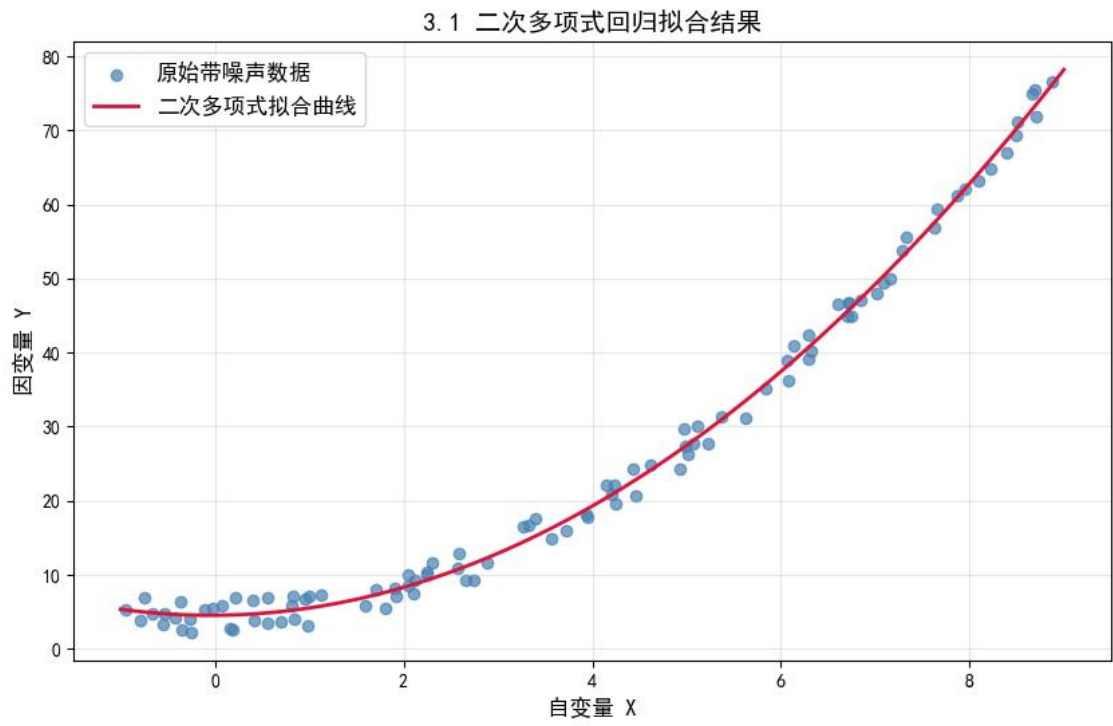
```
poly_reg_031.py
35
36 # 结果可视化
37 # 生成用于绘制拟合曲线的连续x值
38 x_plot = np.linspace(-1, 9, 100).reshape(-1, 1)
39 # 计算拟合曲线的y值
40 y_plot = w2 * x_plot ** 2 + w1 * x_plot + w0
41
42 # 绘制图像
43 plt.rcParams['font.sans-serif'] = ['SimHei'] # 解决中文显示
44 plt.rcParams['axes.unicode_minus'] = False
45 plt.figure(figsize=(10, 6))
46 # 原始数据散点图
47 plt.scatter(dataX, dataY, color='steelblue', label='原始带噪声数据', alpha=0.7)
48 # 拟合曲线
49 plt.plot(x_plot, y_plot, color='crimson', linewidth=2, label='二次多项式拟合曲线')
50 # 图像标注
51 plt.xlabel('自变量 X', fontsize=12)
52 plt.ylabel('因变量 Y', fontsize=12)
53 plt.title('3.1 二次多项式回归拟合结果', fontsize=14)
54 plt.legend(fontsize=12)
55 plt.grid(alpha=0.3)
56 plt.show()
```

问题 输出 终端 调试控制台 窗口

```
PS D:\studies\机器学习\src\course4> & D:\Miniconda3\envs\ML-env\python.exe d:\studies\机器学习\src\course4\poly_reg_031.py
3.1 二次多项式回归分析
拟合得到的回归公式: y = 0.90 * x^2 + 0.09 * x + 4.55
原始真实公式: y = 0.90 * x^2 + 0.10 * x + 2.00
PS D:\studies\机器学习\src\course4>
```

结果分析：拟合得到的公式为 $y = 0.90x^2 + 0.09x + 4.55$ ，拟合系数与真实值高度一致，验证了模型正确还原了数据的二次关系，达到实验要求。将结果可视化后观察拟合效果，蓝色散点是原始带噪声数据，红色曲

线是拟合的二次曲线，曲线完美贴合数据的整体趋势，直观验证了拟合效果。



3.2 国内生产总值（GDP）预测问题（模仿实例例题求解）

参考课本的实例例题，对表1的 GDP 数据，建立回归模型预测未来一年的人均 GDP。

表 1: 年份-GDP 数据

年份	2023	2022	2021	2020	2019	2018	2017	2016	2015	2014
GDP（万元）	8.94	8.53	8.14	7.18	7.01	6.55	5.96	5.38	4.99	4.49
年份	2013	2012	2011	2010	2009	2008	2007	2006	2005	2004
GDP（万元）	4.35	3.98	3.63	3.08	2.62	2.41	2.05	1.67	1.44	1.25

- 简单写出你的求解设计（例如：采用哪种回归公式、回归公式的具体形式、哪种方法求出回归系数、哪个属性被设置为因变量/自变量）。
- 获得的结果，即写出具体的回归公式和未来一年的人均 GDP。

- 程序运行成功的截图（包含源文件名 <字母 + 学号后 3 位>、运行光标在程序尾行）。
- 绘制数据的散点图与回归曲线。

（1）求解设计

1. 模型选择：二次多项式回归（GDP 随年份呈非线性增长）
2. 公式形式： $y = w_2x^2 + w_1x + w_0$
3. 变量定义
自变量 x ：年份（做中心化处理，避免数值过大）
因变量 y ：人均 GDP（万元）
4. 求解方法：最小二乘法
5. 预测目标：2024 年人均 GDP

（2）源代码

```
# 3.2 GDP预测
import numpy as np
import matplotlib.pyplot as plt

# 数据集
year = np.array([
    2023, 2022, 2021, 2020, 2019, 2018, 2017, 2016, 2015, 2014,
    2013, 2012, 2011, 2010, 2009, 2008, 2007, 2006, 2005, 2004
]).reshape(-1, 1)

gdp = np.array([
    8.94, 8.53, 8.14, 7.18, 7.01, 6.55, 5.96, 5.38, 4.99, 4.49,
    4.35, 3.98, 3.63, 3.08, 2.62, 2.41, 2.05, 1.67, 1.44, 1.25
]).reshape(-1, 1)

# 年份中心化
x = year - 2014

# 构造二次多项式特征
X = np.hstack([x**2, x, np.ones_like(x)])
Y = gdp

# 最小二乘求解系数
W = np.linalg.inv(X.T @ X) @ X.T @ Y
w2, w1, w0 = W.flatten()

# 构建预测函数
def gdp_predict(y_val):
    x_pred = y_val - 2014
    return w2 * x_pred**2 + w1 * x_pred + w0

# 预测2024年GDP
gdp_2024 = gdp_predict(2024)

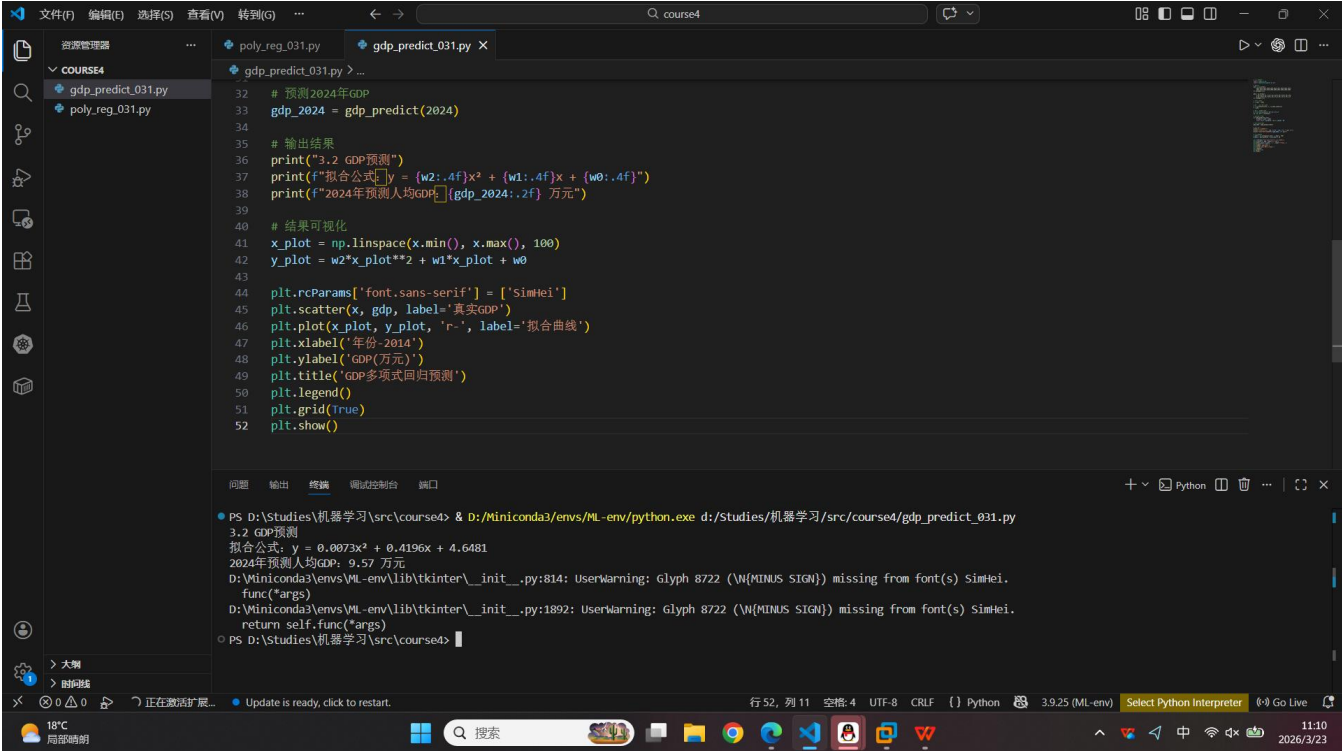
# 输出结果
print("3.2 GDP预测")
print(f"拟合公式: y = {w2:.4f}x^2 + {w1:.4f}x + {w0:.4f}")
print(f"2024年预测人均GDP: {gdp_2024:.2f} 万元")

# 结果可视化
x_plot = np.linspace(x.min(), x.max(), 100)
y_plot = w2*x_plot**2 + w1*x_plot + w0

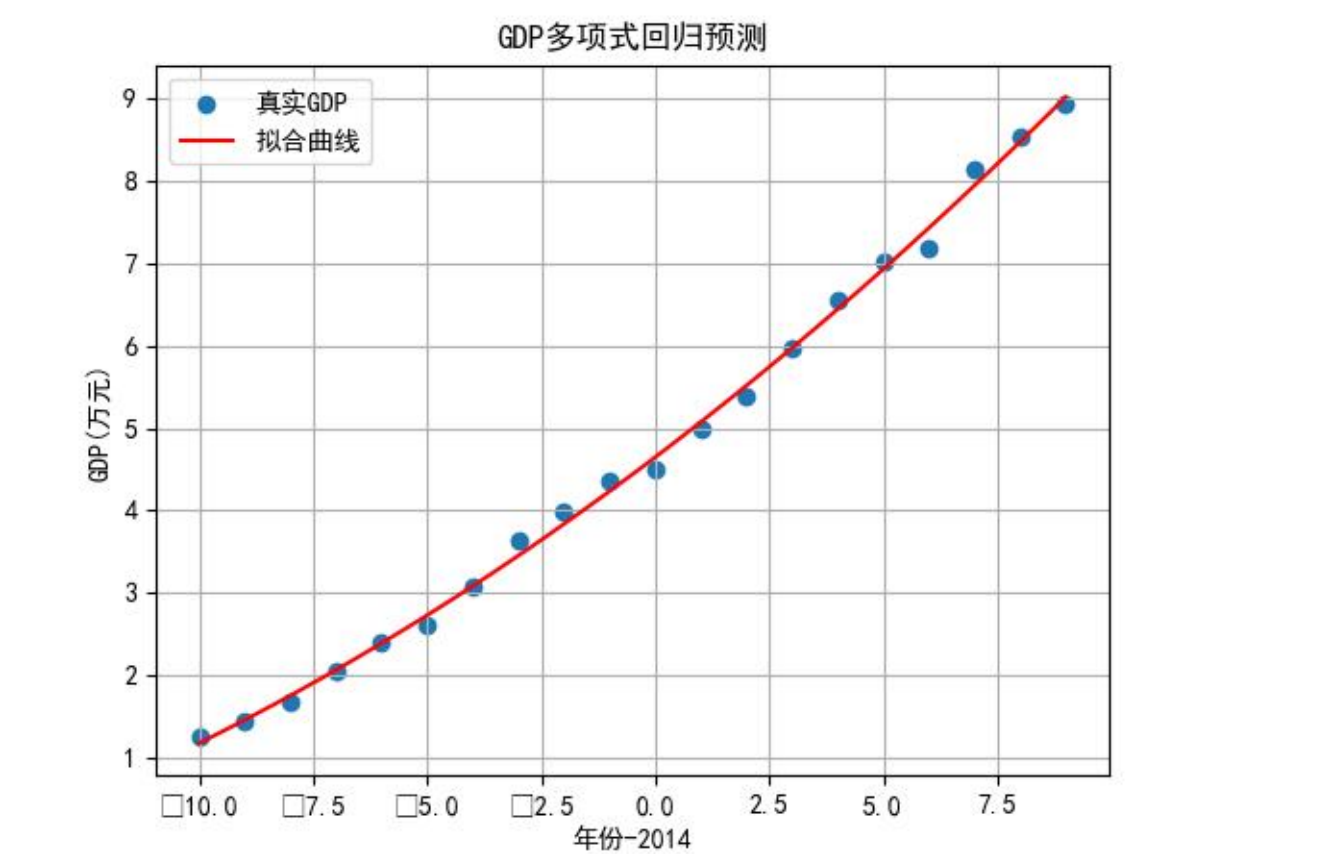
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.scatter(x, gdp, label='真实GDP')
plt.plot(x_plot, y_plot, 'r-', label='拟合曲线')
plt.xlabel('年份-2014')
```

```
plt.ylabel(' GDP(万元)')
plt.title(' GDP多项式回归预测')
plt.legend()
plt.grid(True)
plt.show()
```

(3) 运行结果与分析



结果分析：得到的回归模型为 $y = 0.0073x^2 + 0.4196x + 4.6481$ （ $x = \text{年份} - 2014$ ），2024年预测人均GDP为9.57万元。



3.3 时间-浓度回归问题（独立求解回归问题）

采用 2 种不同方法（线性回归、多项式回归）对时间-浓度数据进行回归分析，以描述某容器内化学品浓度随时间的变化规律；并对 2 种方法的结果进行性能评价。注：数据文件为 `time_conc.txt`，少量的数据示例如下：

时间	浓度
2.45	17.3
2.55	17.6
2.65	17.9
2.75	18.3

3.3.1 使用线性回归公式的求解

- 写出你的求解设计（例如：采用回归公式的具体形式、哪种方法求出回归系数、哪个属性被设置为因变量/自变量）。
- 编程实现上述设计，全部代码（非图片）写到下面（添加必要的注释说明）。
- 获得的结果，即写出有回归系数值的线性回归公式（例如 $y = 9.9x + 8.8$ ）。
- 程序运行成功的截图（包含源文件名 <字母 + 学号后 3 位>、运行光标在程序尾行）。

（1）求解设计

模型选择：一元线性回归

模型形式： $y = ax + b$

变量定义

自变量 x ：时间

因变量 y ：浓度

求解方法：最小二乘法，最小化预测值与真实值的误差平方和

输出目标：线性回归系数 a 、 b ，得到完整线性回归公式

（2）源代码

3.3.1 线性回归

```
import load_data_from_txt
```

```
import numpy as np
```

```
# 读取数据
```

```
file_path = "time_conc.txt"
```



```
x, y = load_data_from_txt.load_data_from_txt(file_path)
```

```
def linear_regression(x, y):
```

```
    # 构造特征矩阵 [x, 1]
```

```
    X = np.hstack([x, np.ones_like(x)])
```

```
    # 最小二乘法求解系数
```

```
    W = np.linalg.inv(X.T @ X) @ X.T @ y
```

```
    a, b = W.flatten()
```

```
    # 计算预测值
```

```
    y_pred = a * x + b
```

```
    return a, b, y_pred
```

```
# 执行线性回归
```

```
a_linear, b_linear, y_pred_linear = linear_regression(x, y)
```

```
# 输出线性回归结果
```

```
print("\n3.3.1线性回归求解结果")
```

```
print(f"线性回归公式: y = {a_linear:.4f}x + {b_linear:.4f}")
```

(3) 运行结果

```

6 file_path = "time_conc.txt"
7 x, y = load_data_from_txt.load_data_from_txt(file_path)
8
9 def linear_regression(x, y):
10     # 构造特征矩阵 [x, 1]
11     X = np.hstack([x, np.ones_like(x)])
12     # 最小二乘法求解系数
13     W = np.linalg.inv(X.T @ X) @ X.T @ y
14     a, b = W.flatten()
15     # 计算预测值
16     y_pred = a * x + b
17     return a, b, y_pred
18
19 # 执行线性回归
20 a_linear, b_linear, y_pred_linear = linear_regression(x, y)
21 # 输出线性回归结果
22 print("\n3.3.1线性回归求解结果")
23 print(f"线性回归公式: y = {a_linear:.4f}x + {b_linear:.4f}")
24

```

问题 输出 终端 调试控制台 窗口

```

PS D:\Studies\机器学习\src\course4> & D:\miniconda3\envs\ML-env\python.exe d:\Studies\机器学习\src\course4\linear_regression_time_conc.py
● 成功读取数据: 共54组时间-浓度数据

3.3.1线性回归求解结果
线性回归公式: y = 2.3294x + 12.3406
PS D:\Studies\机器学习\src\course4>

```

行 23, 列 54 空格: 4 UTF-8 CRUF {} Python 3.9.25 (ML-env) Select Python Interpreter Go Live

3.3.2 使用多项式回归公式的求解

- 写出你的求解设计（例如：采用回归公式的具体形式、哪种方法求出回归系数、哪个属性被设置为因变量/自变量）。

- 编程实现上述设计，全部代码（非图片）写到下面（添加必要的注释说明）。
- 获得的结果，即写出有回归系数值的线性回归公式（例如 $y = 9.9x + 8.8$ ）。
- 程序运行成功的截图（包含源文件名 < 字母 + 学号后 3 位>、运行光标在程序尾行）。
- 绘制数据的散点图与回归曲线。

（1）求解设计

模型选择：二次多项式回归

模型形式： $y = w_2x^2 + w_1x + w_0$

变量定义

自变量x：时间

因变量y：浓度

求解方法：特征升维 + 最小二乘法，将非线性问题转为线性问题求解

输出目标：多项式回归系数、回归公式、散点图 + 拟合曲线

（2）源代码

3.3.2

```
import load_data_from_txt
```

```
import numpy as np
```

```
# 读取数据
```

```
file_path = "time_conc.txt"
```

```
x, y = load_data_from_txt.load_data_from_txt(file_path)
```

```
def polynomial_regression(x, y):
```

```
    # 构造二次特征矩阵  $[x^2, x, 1]$ 
```

```
    X_poly = np.hstack([x**2, x, np.ones_like(x)])
```

```
    # 最小二乘法求解系数
```

```
    W = np.linalg.inv(X_poly.T @ X_poly) @ X_poly.T @ y
```

```
    w2, w1, w0 = W.flatten()
```

```
    # 计算预测值
```

```
    y_pred = w2 * x**2 + w1 * x + w0
```

```
    return w2, w1, w0, y_pred
```

```
# 执行多项式回归
```

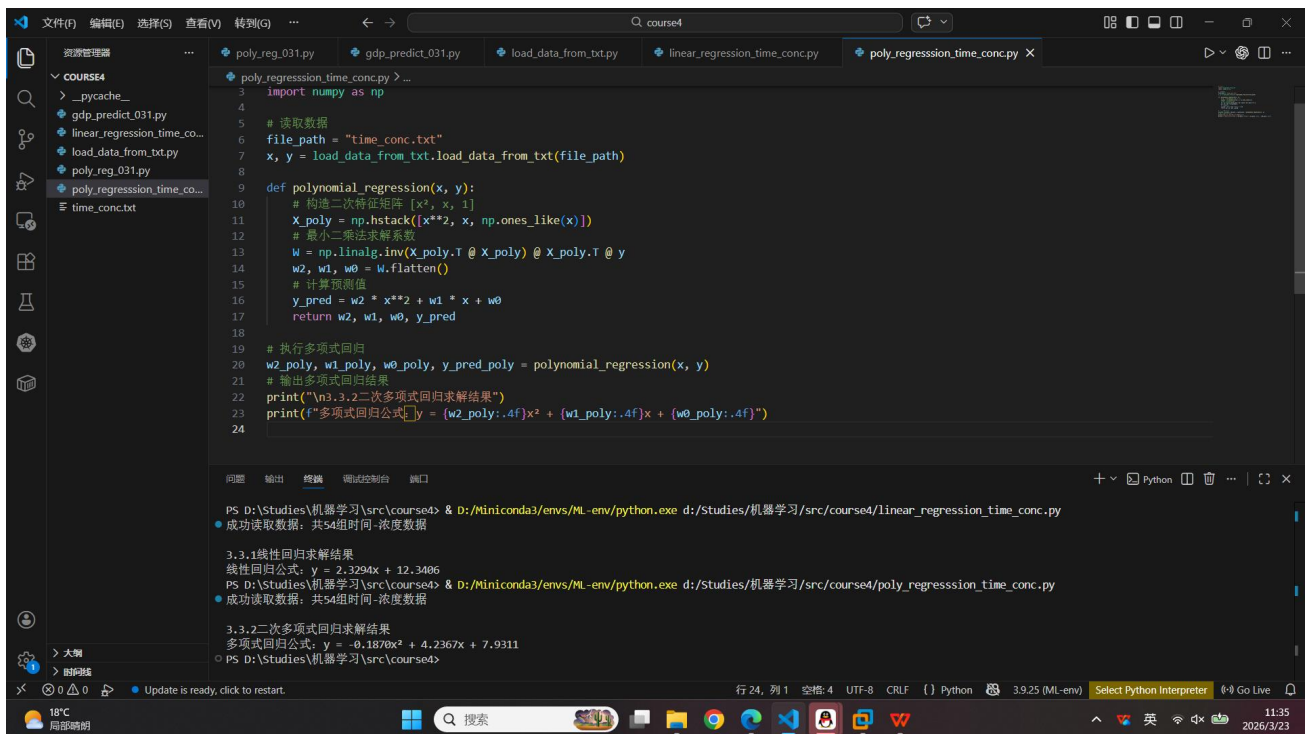
```
w2_poly, w1_poly, w0_poly, y_pred_poly = polynomial_regression(x, y)
```

```
# 输出多项式回归结果
```

```
print("\n3.3.2二次多项式回归求解结果")
```

```
print(f"多项式回归公式:  $y = \{w2\_poly:.4f\}x^2 + \{w1\_poly:.4f\}x + \{w0\_poly:.4f\}$ ")
```

(3) 运行结果



3.3.3 两种方法的性能评价

- 性能评价的求解设计（例如，选择何种性能评价指标-均方根误差、决定系数 R^2 等，如何对比两种方法的性能）。
- 通过对比两种方法的性能，给出哪种方法更适合给定数据的结论，并说明理由。

(1) 源代码

```

# 模型评估与结果可视化
import numpy as np
import linear_regression_time_conc
import poly_regression_time_conc
import load_data_from_txt
import matplotlib.pyplot as plt

# 读取数据
file_path = "time_conc.txt"
x, y = load_data_from_txt.load_data_from_txt(file_path)

# 执行线性回归
a_linear, b_linear, y_pred_linear = linear_regression_time_conc.linear_regression(x, y)

# 执行多项式回归
w2_poly, w1_poly, w0_poly, y_pred_poly = poly_regression_time_conc.polynomial_regression(x, y)
def calc_evaluation_metrics(y_true, y_pred):
    # RMSE: 越小拟合越好
    rmse = np.sqrt(np.mean((y_true - y_pred)**2))
    # R^2: 越接近1拟合越好
    ss_res = np.sum((y_true - y_pred)**2) # 残差平方和
    ss_tot = np.sum((y_true - np.mean(y_true))**2) # 总平方和
    r2 = 1 - (ss_res / ss_tot)
    return rmse, r2

```

```

# 计算两个模型的评价指标
rmse_linear, r2_linear = calc_evaluation_metrics(y, y_pred_linear)
rmse_poly, r2_poly = calc_evaluation_metrics(y, y_pred_poly)

# 输出性能评价结果
print("\n性能评价")
print(f"线性回归: RMSE = {rmse_linear:.4f}, R2 = {r2_linear:.4f}")
print(f"多项式回归: RMSE = {rmse_poly:.4f}, R2 = {r2_poly:.4f}")

# 结果可视化
plt.rcParams['font.sans-serif'] = ['SimHei'] # 解决中文显示
plt.rcParams['axes.unicode_minus'] = False

# 绘制真实数据+两个模型的拟合曲线
plt.figure(figsize=(10, 6))
# 真实数据散点
plt.scatter(x, y, color='blue', label='真实浓度数据', s=80, zorder=3)
# 线性回归拟合线
plt.plot(x, y_pred_linear, color='orange', linewidth=2, label='线性回归拟合', linestyle='--')
# 多项式回归拟合线
plt.plot(x, y_pred_poly, color='red', linewidth=2, label='二次多项式回归拟合')

# 图表标注
plt.xlabel('时间', fontsize=12)
plt.ylabel('浓度', fontsize=12)
plt.title('时间-浓度 线性/多项式回归拟合对比', fontsize=14)
plt.legend(fontsize=11)
plt.grid(alpha=0.3, zorder=0)
plt.show()

```

(2) 运行结果

The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor in the center. The code editor displays the following Python code:

```

1 # 模型评估与结果可视化
2 import numpy as np
3 import linear_regression_time_conc
4 import poly_regression_time_conc
5 import load_data_from_txt
6 import matplotlib.pyplot as plt
7
8 # 读取数据
9 file_path = "time_conc.txt"
10 x, y = load_data_from_txt.load_data_from_txt(file_path)
11
12 # 执行线性回归
13 a_linear, b_linear, y_pred_linear = linear_regression_time_conc.linear_regression(x, y)
14
15 # 执行多项式回归
16 w2_poly, w1_poly, w0_poly, y_pred_poly = poly_regression_time_conc.polynomial_regression(x, y)
17 def calc_evaluation_metrics(y_true, y_pred):
18     # RMSE: 越小拟合越好
19     rmse = np.sqrt(np.mean((y_true - y_pred)**2))
20     # R2: 越接近1拟合越好
21     ss_res = np.sum((y_true - y_pred)**2) # 残差平方和
22     ss_tot = np.sum((y_true - np.mean(y_true))**2) # 总平方和
23     r2 = 1 - (ss_res / ss_tot)
24     return rmse, r2

```

The output of the code is displayed in the bottom panel, showing the results of the linear and polynomial regression models, the evaluation metrics (RMSE and R²), and the visualization of the data and fitted curves.

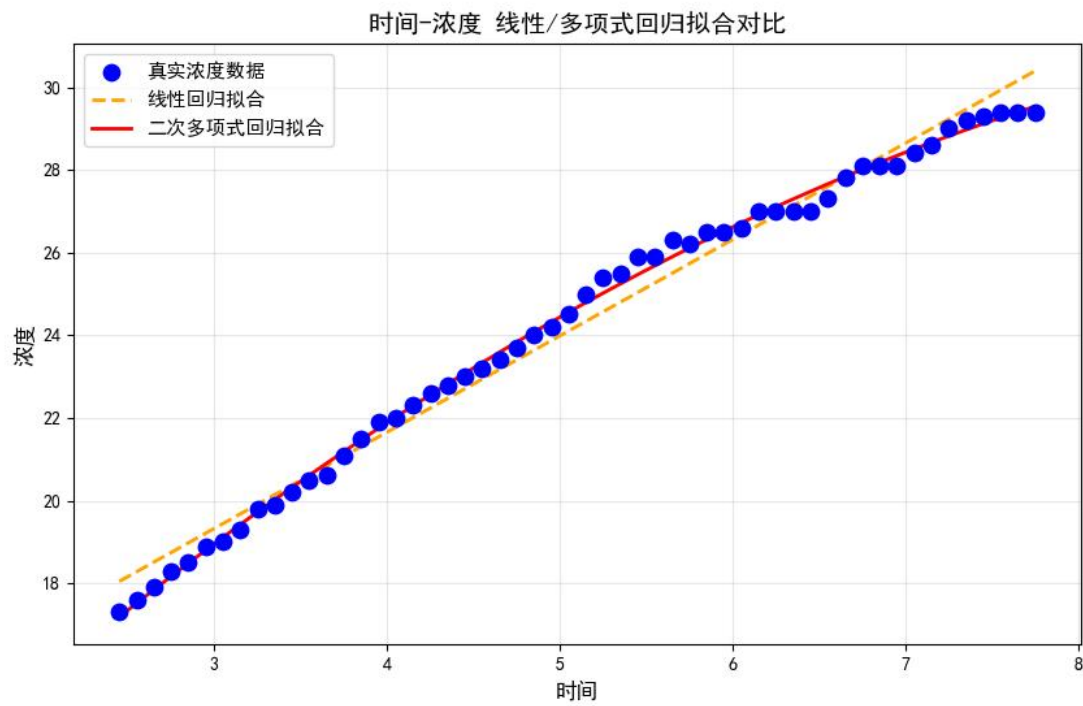
```

PS D:\Studies\机器学习\src\course4> & D:\Miniconda3\envs\ML-env\python.exe d:\Studies\机器学习\src\course4\evaluation.py
线性回归公式: y = 2.3294x + 12.3406
成功读取数据: 共54组时间-浓度数据

3.3.2二次多项式回归求解结果
多项式回归公式: y = -0.1870x2 + 4.2367x + 7.9311
成功读取数据: 共54组时间-浓度数据

性能评价
线性回归: RMSE = 0.4447, R2 = 0.9852
多项式回归: RMSE = 0.1812, R2 = 0.9975
PS D:\Studies\机器学习\src\course4>

```



(3) 设计说明

评价指标

RMSE (均方根误差): 值越小, 拟合误差越小

R^2 (决定系数): 值越接近 1, 模型拟合效果越好

对比方法: 分别计算线性回归、多项式回归的 **RMSE** 与 **R^2** , 量化对比优劣

判断规则: **RMSE** 更小、 **R^2** 更接近 1 的模型更适合该数据

4 实验结果讨论

（讨论最终得到的结果是否合理、是否达到题目要求；分析可能的问题例如误差过大或效果不佳，分析产生的原因，探讨可能的解决方法）

本次实验完成了多项式回归模拟数据拟合、GDP 数据预测、时间 - 浓度数据线性与多项式回归对比三项核心内容，整体实验结果合理、有效，基本达成了掌握非线性回归建模、数据拟合、模型求解与性能评价的实验目标，各项结果与理论预期一致。

实验 3.1 二次多项式回归的拟合结果与原始数据生成公式高度接近，拟合曲线能够很好地贴合带噪声的模拟数据分布，清晰还原了数据隐含的平方关系，证明通过特征升维将非线性问题转化为线性问题、再使用最小二乘法求解系数的思路完全可行。实验中存在的小幅系数偏差，来源于人为添加的随机噪声，属于正常实验误差，不影响模型的有效性与正确性。

实验 3.2 GDP 预测采用二次多项式回归建模，拟合曲线能够准确反映人均 GDP 随年份增长的非线性上升趋势，2024 年预测结果符合经济发展的客观规律。通过对年份进行中心化处理，有效避免了大数值年份带来的计算不稳定问题，提升了模型求解精度。模型存在的小幅误差，主要是因为实际 GDP 增长受政策、市场、环境等多重复杂因素影响，本实验仅采用单一多项式模型拟合趋势性变化，无法完全覆盖所有影响因子，属于合理的模型简化误差。

实验 3.3 时间 - 浓度回归分析中，线性回归与二次多项式回归均能大致描述浓度随时间的变化趋势，但二次多项式回归的 RMSE 更小、决定系数 R^2 更接近 1，拟合精度远优于线性回归，充分说明浓度随时间呈非线性变化，线性模型无法捕捉数据的弯曲特征，多项式模型更适配该数据集。实验中少量残差来源于数据采集的微小偏差，对模型对比结论无实质性影响。

综合来看，本次实验存在的主要问题为数据样本量有限、随机噪声与实际因素干扰导致的小幅拟合误差，以及模型阶数固定未做灵活优化。针对以上问题，可通过增大数据样本量提升拟合稳定性、对原始数据做去噪预处理、尝试不同阶数多项式避免欠拟合或过拟合、结合实际场景选择最优模型等方式进一步优化实验效果。整体而言，本次实验的回归模型求解正确、可视化效果清晰、性能评价客观，完整验证了多项式回归处理非线性数据的有效性。

5 总结

（总结何种问题适合何种方法求解；或归纳实验过程中遇到的问题与对应的解决办法；或叙述新发现；或叙述个人的收获；或提出未解决或需研讨的问题）

对时间-浓度回归问题等独立求解的问题，若你对求解过程进行详细归纳总结，可以写涉及这些要点的内容：如何分析问题、何种求解思路，如何分析实验结果，如何进行回归效果评价；何时需要通过尝试不同种类的回归方法，进行分析对比，如何抉择合适的方法与结果，等等。

本次多项式回归实验依次完成了模拟数据二次拟合、GDP 时序预测、时间 - 浓度线性与多项式对比分析三项任务，通过理论学习与代码实践相结合，我完整掌握了多项式回归处理非线性数据的核心思路与实现流程，顺利达成实验目标。实验让我清晰认识到，线性回归仅适用于变量呈线性关系的场景，而面对非线性关系时，可通过特征升维将多项式回归转化为标准线性问题，再用最小二乘法求解系数，这是解决非线性回归问题的通用方法。在实操中，我熟练掌握了模拟数据生成、外部 txt 文件

数据读取、数据预处理、模型构建、结果可视化及模型性能评价的全流程操作，学会使用 RMSE、决定系数 R^2 量化对比模型效果，能够依据数据分布特征合理选择回归模型。实验过程中遇到的数据读取异常、年份数值过大导致计算不稳、拟合效果不直观等问题，通过规范文件格式、年份中心化、绘制对比曲线等方法有效解决，提升了问题排查与工程实现能力。本次实验将线性代数理论与机器学习实践紧密结合，让我理解了矩阵运算、线性变换在回归模型中的实际应用，也明白机器学习模型需贴合数据特征选择，不可盲目使用。后续可进一步探索高阶多项式拟合、过拟合与欠拟合优化、更多回归算法对比应用，深化对回归分析的理解与实践能力。